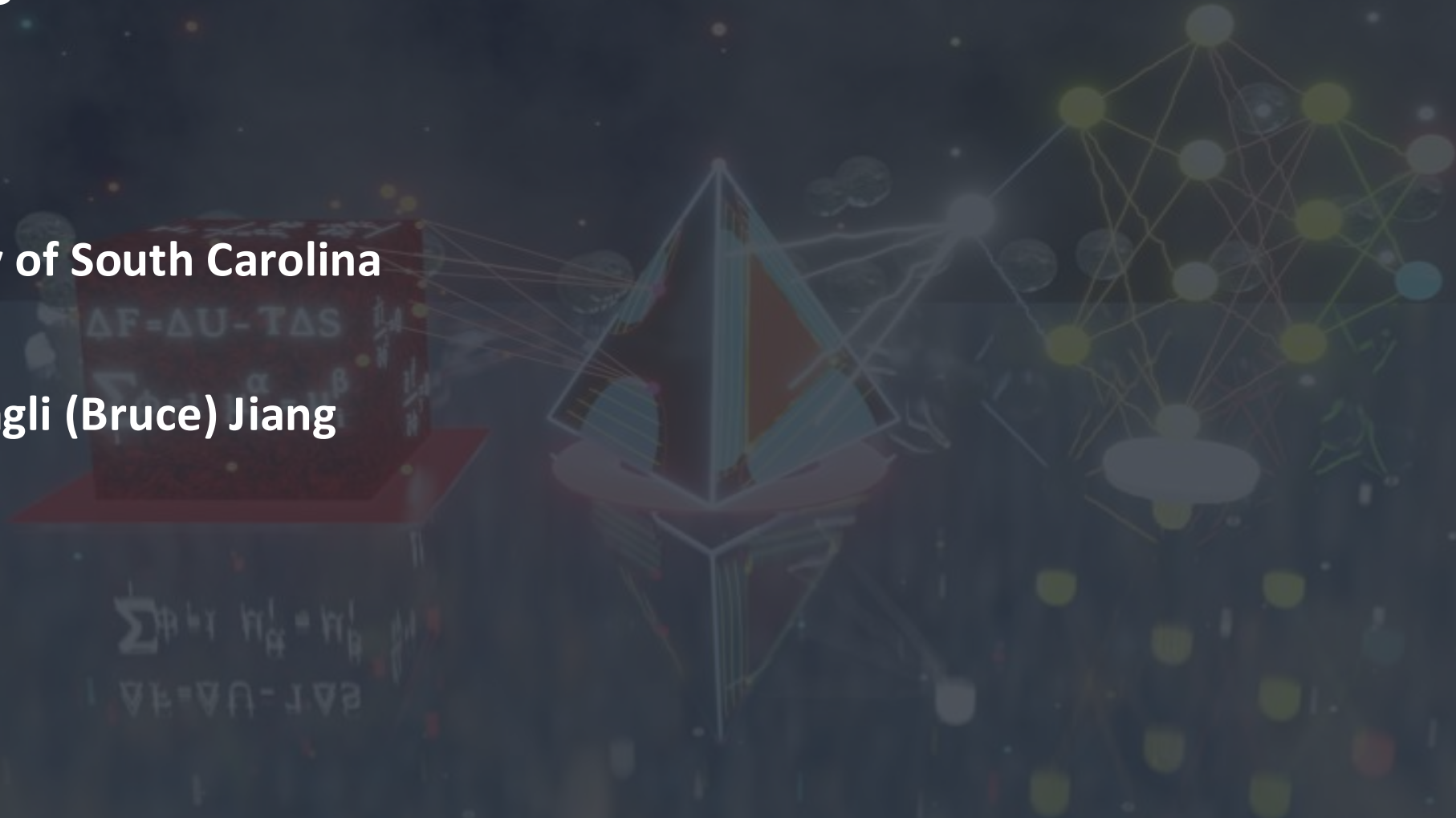


# Classification IV: Decision Trees and Random Forests

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



## **Classification: Kernels and Decision Trees**

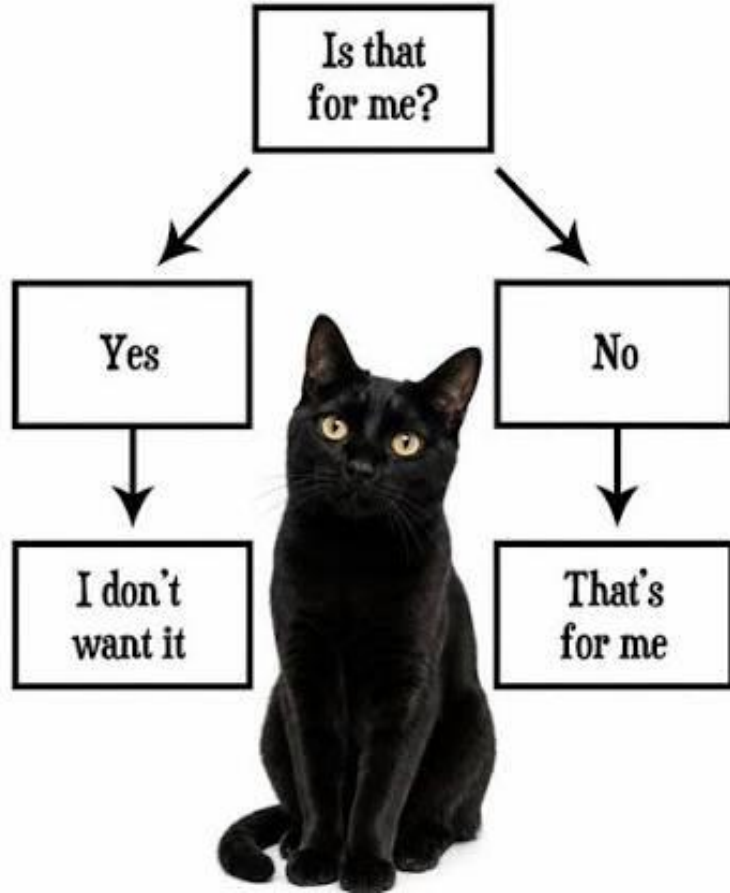
- *Kernels allow SVMs to learn nonlinear decision boundaries.*
- *Polynomial degree and RBF  $\gamma$  control model flexibility.*
- *Decision trees choose splits that reduce Gini impurity.*

### **In this lecture, you will learn**

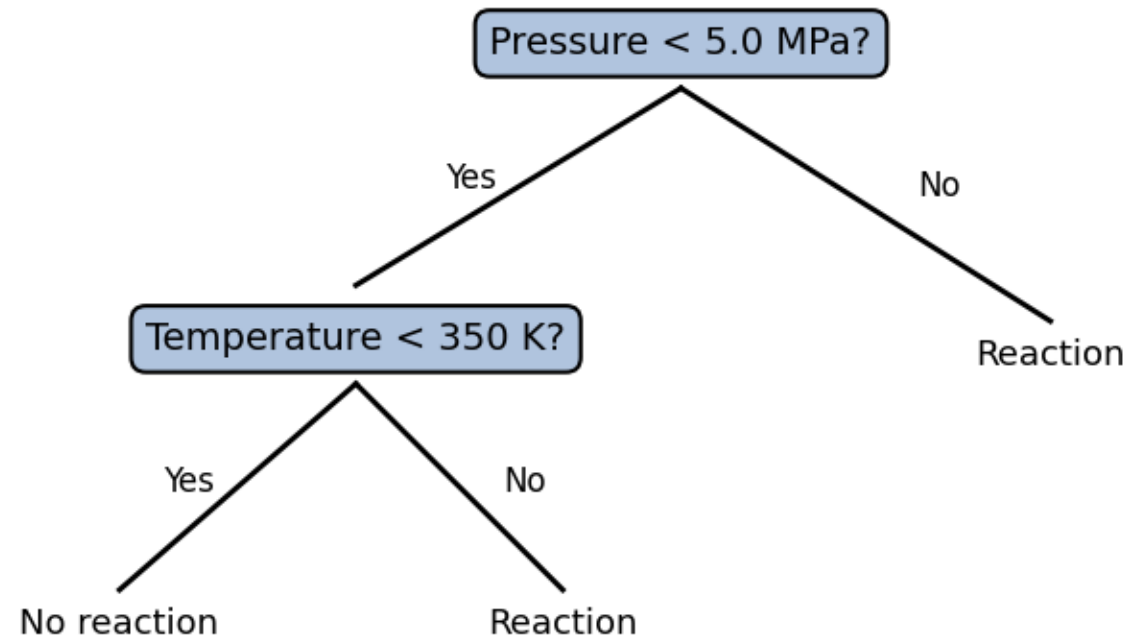
- *- How entropy and information gain measure split quality.*
- *- How stopping rules control tree complexity and overfitting.*
- *- How random forests combine different trees and measure feature importance.*

# Decision Tree

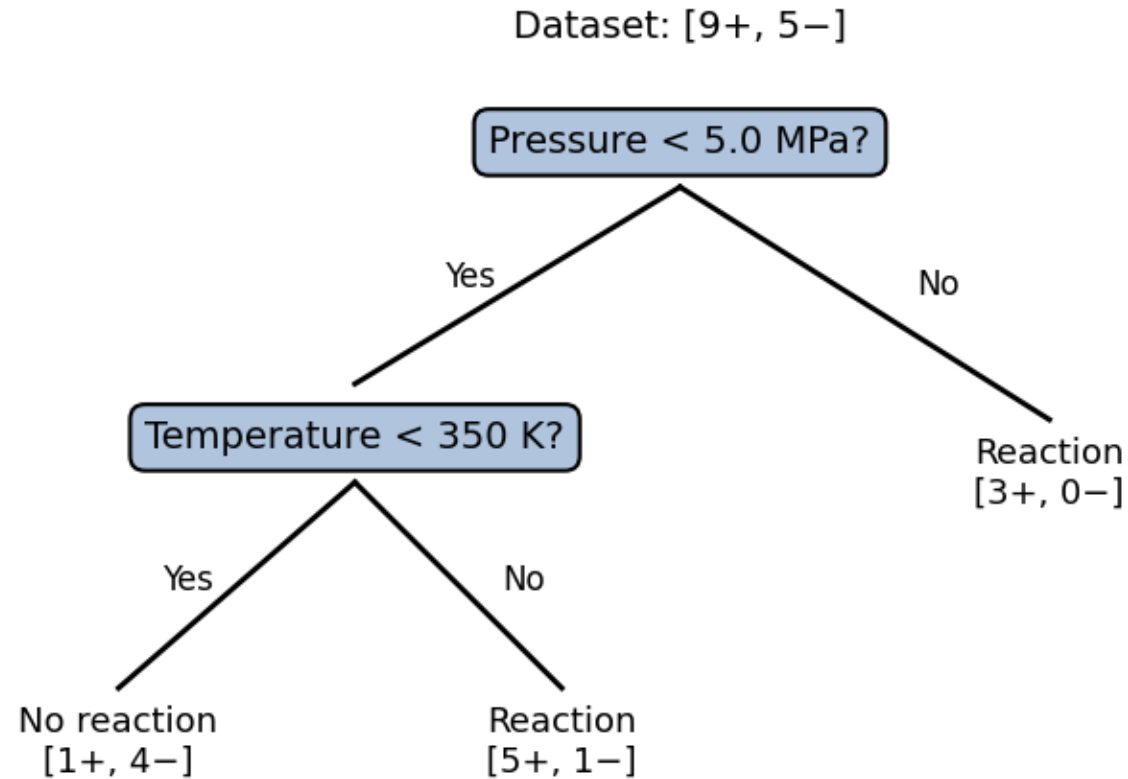
## Cat's Decision Making Tree



A decision tree partitions the feature space using simple if-then rules.



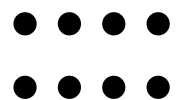
# How Does a Tree Split the Data?



- The algorithm considers many possible features and thresholds.
  - Pressure < 2.0 MPa, Pressure < 3.0 MPa, Temperature < 340 K
- **Which split makes the resulting groups as pure as possible?**

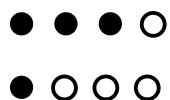
# What is a Good Split?

Node A



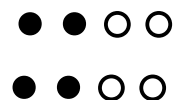
pure

Node B



mixed

Node C



very mixed

For a dataset  $D$  with  $k$  classes, the **Gini impurity** is

$$G = \text{Gini}(D) = 1 - \sum_{i=1}^k p_i^2$$

For binary classification:

$$G = 1 - (p_0^2 + p_1^2)$$

Pure node:

$$p_0 = 1, p_1 = 0$$

$$G = 1 - (1^2 + 0^2) = 0$$

50/50 node:

$$p_0 = 0.5, p_1 = 0.5$$

$$G = 1 - (0.5^2 + 0.5^2) = 0.5$$

**Lower impurity = cleaner separation of classes.**

# Finding the Best Split

Parent

● ● ● ○ ○ ○

Candidate split A:

Left

● ● ○

Right

● ○ ○

Candidate split B:

Left

● ● ●

Right

○ ○ ○

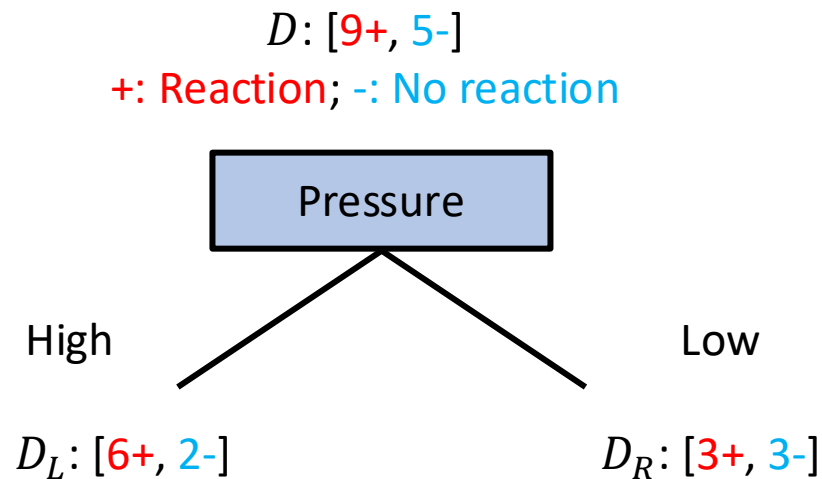
$$G_{\text{split}} = \text{weighted impurity after split} = \frac{|D_L|}{|D|} G_L + \frac{|D_R|}{|D|} G_R$$

- $|D|$ : number of samples in the parent node
- $|D_L|$ : number of samples that go to the **left child**
- $|D_R|$ : number of samples that go to the **right child**
- $|D| = |D_L| + |D_R|$

**Choose the split with the largest impurity reduction.**

**Which split is better?**

# Example



$$\text{Gini}(D_L) = 1 - \left(\frac{6}{8}\right)^2 - \left(\frac{2}{8}\right)^2 = 0.375$$

$$\text{Gini}(D) = 1 - \sum_{i=1}^k p_i^2$$

$$\text{Gini}_{\text{split}}(D) = \frac{|D_L|}{|D|} \text{Gini}(D_L) + \frac{|D_R|}{|D|} \text{Gini}(D_R)$$

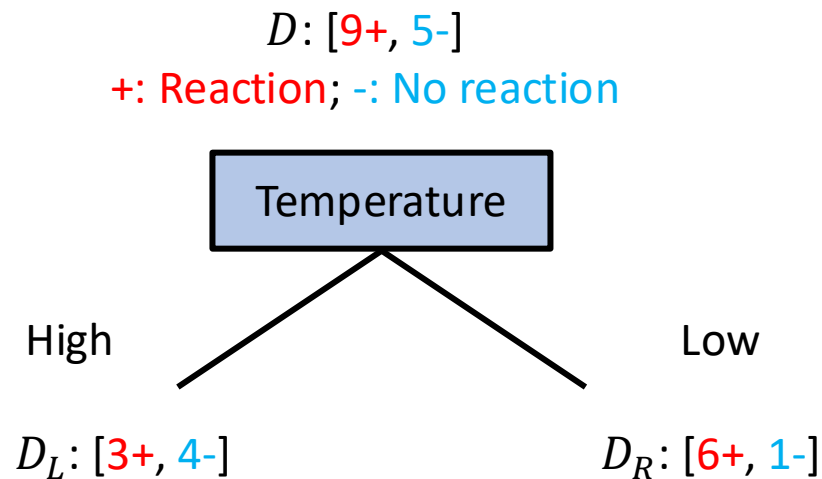
$$\text{Gini}(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

$$\text{Gini}(D_R) = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 0.5$$

$$\text{Gini}_{\text{split}}(D) = \frac{8}{14} \cdot 0.375 + \frac{6}{14} \cdot 0.5 = 0.429$$

$$\text{Gini reduction} = 0.459 - 0.429 = 0.030$$

# Example



$$\text{Gini}(D) = 1 - \sum_{i=1}^k p_i^2$$

$$\text{Gini}_{\text{split}}(D) = \frac{|D_L|}{|D|} \text{Gini}(D_L) + \frac{|D_R|}{|D|} \text{Gini}(D_R)$$

$$\text{Gini}(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

$$\text{Gini}(D_L) = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 0.490$$

$$\text{Gini}(D_R) = 1 - \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 = 0.245$$

$$\text{Gini}_{\text{split}}(D) = \frac{7}{14} \cdot 0.490 + \frac{7}{14} \cdot 0.245 = 0.368$$

$$\text{Gini reduction} = 0.459 - 0.368 = 0.091$$

Is it better to split on pressure or temperature?

# Information Gain / Entropy

Node A

● ● ● ●  
● ● ● ●

pure

Node B

● ● ● ○  
● ○ ○ ○

mixed

Node C

● ● ○ ○  
● ● ○ ○

very mixed

The **Entropy** is a measure of the uncertainty or information content associated with a random variable.

For binary classification:

$$H(X) = - \sum_{x \in X} P(x) \log_2 P(x)$$

$$0 \log_2 0 = 0$$

Pure node:

$$p_0 = 1, p_1 = 0$$

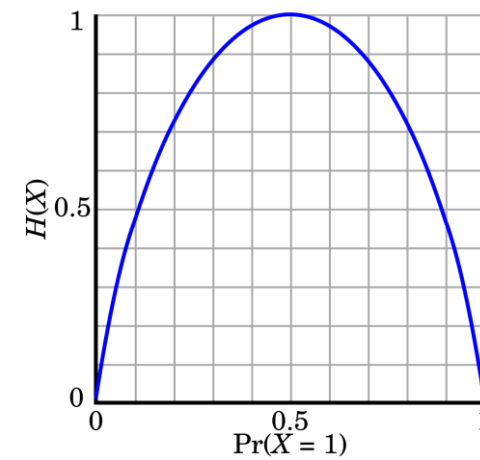
$$H = -(1 \log_2 1 + 0 \log_2 0) = 0$$

50/50 node:

$$p_0 = 0.5, p_1 = 0.5$$

$$H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$$

**Entropy is maximized when all outcomes are equiprobable.**



# Finding the Best Split using Information Gain

Parent

● ● ● ○ ○ ○

Candidate split A:

Left      Right

● ● ○      ● ○ ○

Candidate split B:

Left      Right

● ● ●      ○ ○ ○

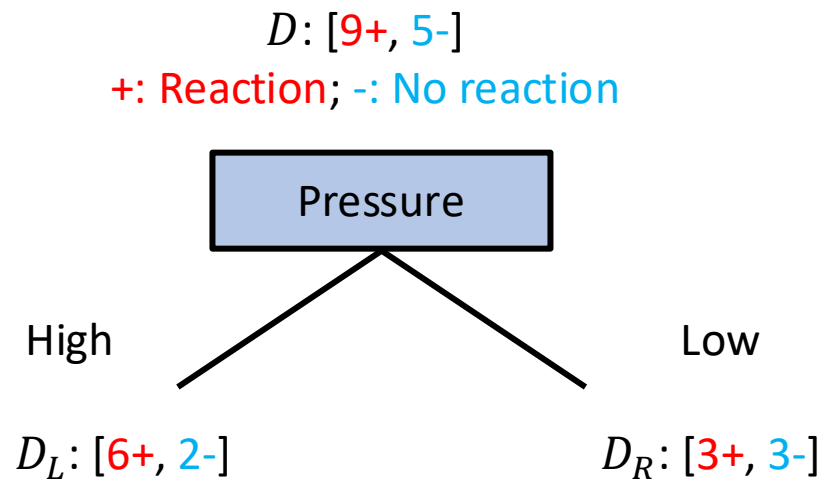
$$H_{\text{split}} = \frac{|D_L|}{|D|} H_L + \frac{|D_R|}{|D|} H_R$$

- $|D|$ : number of samples in the parent node
- $|D_L|$ : number of samples that go to the **left child**
- $|D_R|$ : number of samples that go to the **right child**
- $|D| = |D_L| + |D_R|$

Choose the split with the largest information gain.

Which split is better?

# Information Gain Example



$$H(X) = - \sum_{x \in X} P(x) \log_2 P(x)$$

$$\text{IG}_{\text{split}}(D) = H(D) - \left( \frac{|D_L|}{|D|} H(D_L) + \frac{|D_R|}{|D|} H(D_R) \right)$$

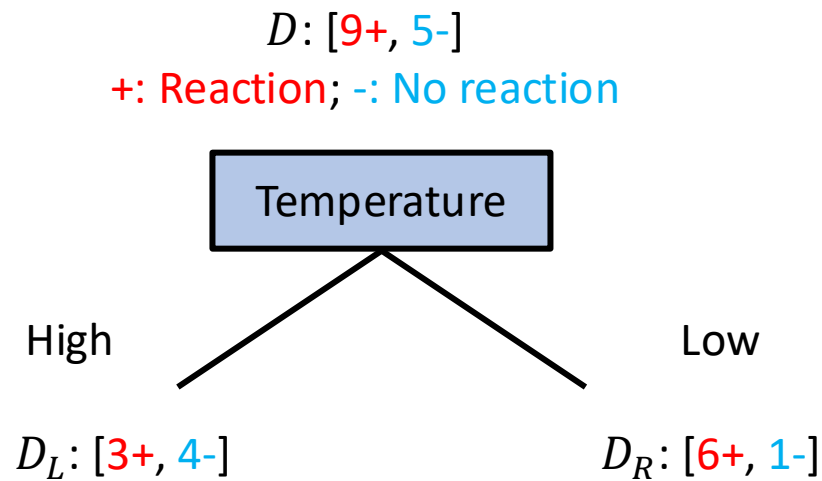
$$H(D) = -\frac{9}{14} \log_2 \left( \frac{9}{14} \right) - \frac{5}{14} \log_2 \left( \frac{5}{14} \right) = 0.940$$

$$H(D_L) = -\frac{6}{8} \log_2 \left( \frac{6}{8} \right) - \frac{2}{8} \log_2 \left( \frac{2}{8} \right) = 0.811$$

$$H(D_R) = -\frac{3}{6} \log_2 \left( \frac{3}{6} \right) - \frac{3}{6} \log_2 \left( \frac{3}{6} \right) = 1.0$$

$$\text{IG}_{\text{split}}(D) = 0.940 - \left( \frac{8}{14} \cdot 0.811 + \frac{6}{14} \cdot 1.0 \right) = 0.048$$

# Information Gain Example



$$H(X) = - \sum_{x \in X} P(x) \log_2 P(x)$$

$$\text{IG}_{\text{split}}(D) = H(D) - \left( \frac{|D_L|}{|D|} H(D_L) + \frac{|D_R|}{|D|} H(D_R) \right)$$

$$H(D) = -\frac{9}{14} \log_2 \left( \frac{9}{14} \right) - \frac{5}{14} \log_2 \left( \frac{5}{14} \right) = 0.940$$

$$H(D_L) = -\frac{3}{7} \log_2 \left( \frac{3}{7} \right) - \frac{4}{7} \log_2 \left( \frac{4}{7} \right) = 0.985$$

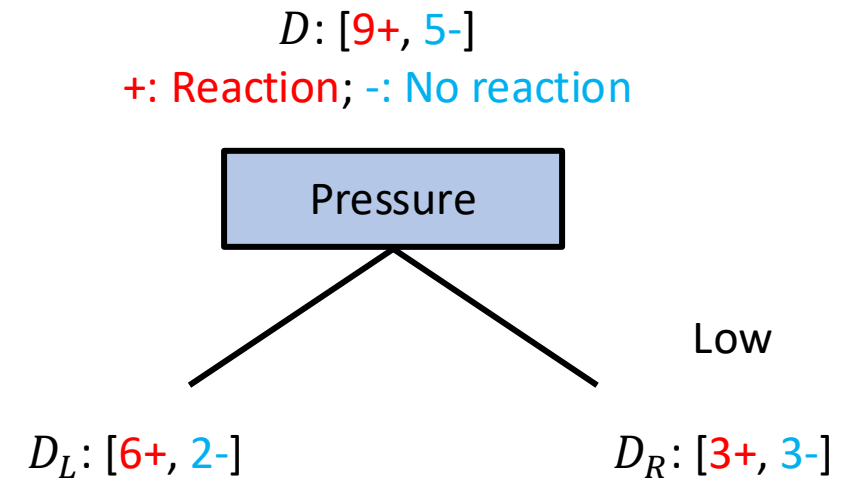
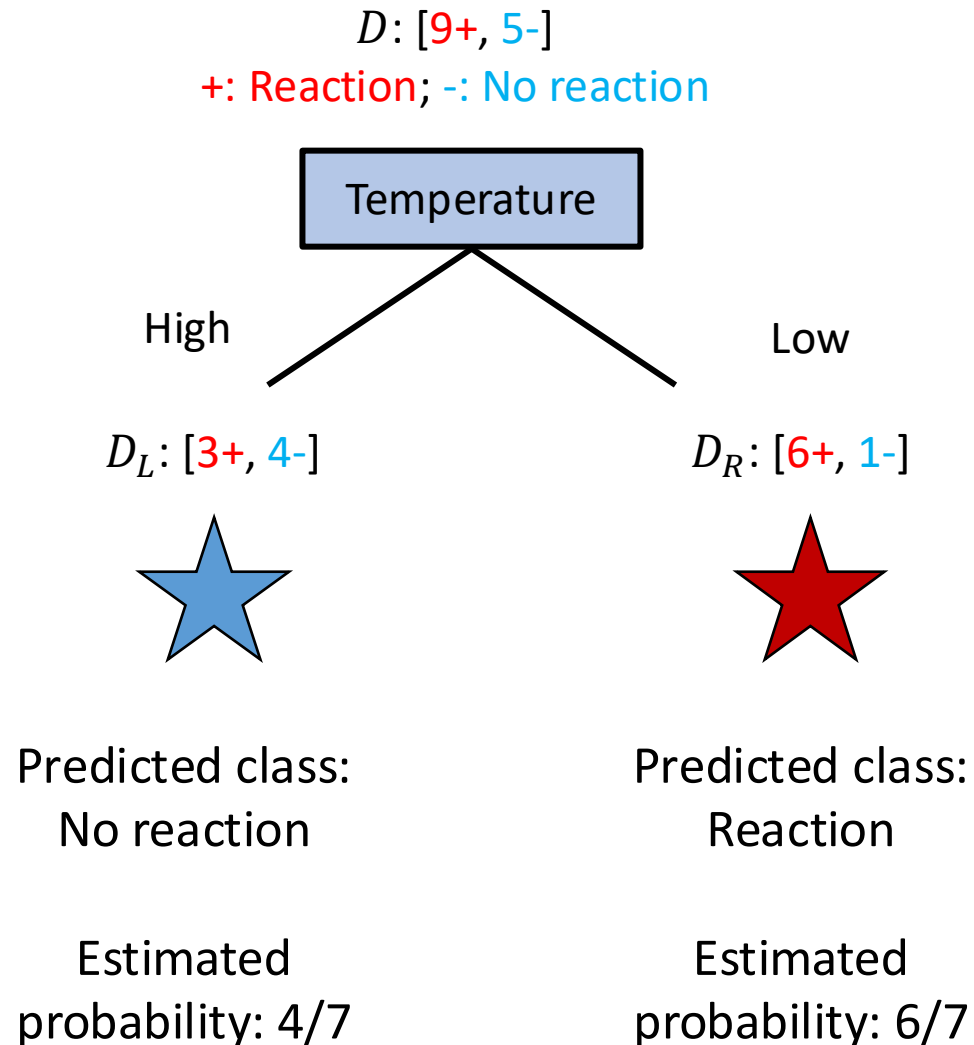
$$H(D_R) = -\frac{6}{7} \log_2 \left( \frac{6}{7} \right) - \frac{1}{7} \log_2 \left( \frac{1}{7} \right) = 0.592$$

$$\text{IG}_{\text{split}}(D) = 0.940 - \left( \frac{7}{14} \cdot 0.985 + \frac{7}{14} \cdot 0.592 \right) = 0.151$$

Is it better to split on pressure or temperature?

# How does a decision tree make a prediction for new data?

- The tree sends the new data point to a leaf, then predicts the majority class in that leaf.
- leaf  $\neq$  necessarily pure



How about this?

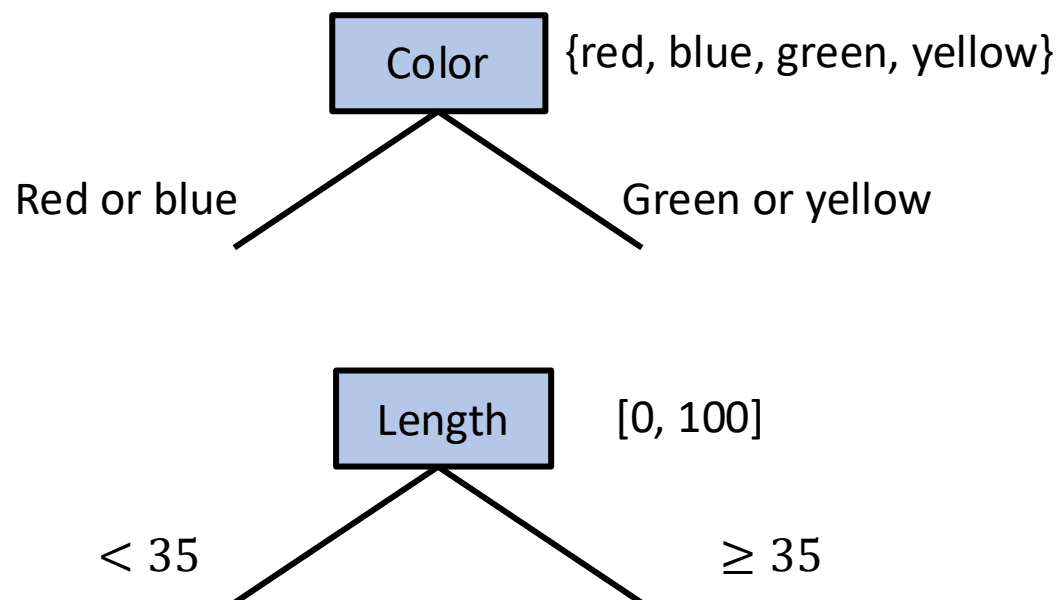
# Best Split and Stopping Criteria

To find the best split at each node:

1. Select a feature.
2. Select a threshold or subset.
3. Maximize the “purity” of the resulting subsets.  
The subsets contain **predominantly** one class.

Common stopping criteria:

- Maximum depth
- Maximum number of leaf nodes
- Minimum samples per leaf
- Minimum samples per split
- Minimum information gain / impurity reduction



Why do we need stopping criteria?

Stopping criteria in decision trees are essential to prevent the tree from becoming overly complex, which can lead to **overfitting**.

# Hyperparameters

## `sklearn.tree.DecisionTreeClassifier`

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best',
max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_features=None, random_state=None, max_leaf_nodes=None,
min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0)
```

[\[source\]](#)

### **`max_depth` : *int*, *default=None***

The maximum depth of the tree. If `None`, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

### **`min_samples_split` : *int or float*, *default=2***

The minimum number of samples required to split an internal node:

- If `int`, then consider `min_samples_split` as the minimum number.
- If `float`, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

**Higher `max_depth`** → more complex tree → higher overfitting risk

**Lower `max_depth`** → simpler tree → possible underfitting

**Higher `min_samples_split`** → A node with 15 samples cannot split → it becomes a leaf earlier → simpler tree.

**Lower `min_samples_split`** → A node with 15 samples can still split → the tree can keep growing deeper → more complex tree.

# Hyperparameters

## sklearn.tree.DecisionTreeClassifier

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best',
max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_features=None, random_state=None, max_leaf_nodes=None,
min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0)
```

[\[source\]](#)

**max\_features** : *int, float or {"auto", "sqrt", "log2"}, default=None*

The number of features to consider when looking for the best split:

- If int, then consider `max_features` features at each split.
- If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split.
- If "sqrt", then `max_features=sqrt(n_features)`.
- If "log2", then `max_features=log2(n_features)`.
- If None, then `max_features=n_features`.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

**random\_state** : *int, RandomState instance or None, default=None*

Controls the randomness of the estimator. The features are always randomly permuted at each split, even if `splitter` is set to "best". When `max_features < n_features`, the algorithm will select `max_features` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if `max_features=n_features`. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, `random_state` has to be fixed to an integer. See [Glossary](#) for details.

**max\_features** controls how many candidate features are examined when choosing each split.

Each individual split still uses only **ONE** feature.

Fix a **random state** for reproducibility.

# Hyperparameter Tuning

## 1. Load data and split

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer(as_frame=True)

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=26
)

print(X.shape)
print(len(X_train), len(X_test))

(569, 30)
455 114
```

## 2. 5-fold CV

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

tree = DecisionTreeClassifier(
    random_state=26
)

param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 5]
}

search = GridSearchCV(
    tree,
    param_grid,
    cv=5,
    scoring="accuracy"
)

search.fit(X_train, y_train)
```

# Hyperparameter Tuning

## 3. Best model and final test

```
from sklearn.metrics import accuracy_score

print("Best CV accuracy:", search.best_score_)
print("Best parameters:", search.best_params_)

y_pred = search.predict(X_test)

print(
    "Test accuracy:",
    accuracy_score(y_test, y_pred)
)
```

Best CV accuracy: 0.9384615384615385

Best parameters: {'criterion': 'entropy', 'max\_depth': 3, 'min\_samples\_leaf': 1, 'min\_samples\_split': 2}

Test accuracy: 0.9473684210526315

## 4. Tree visualization

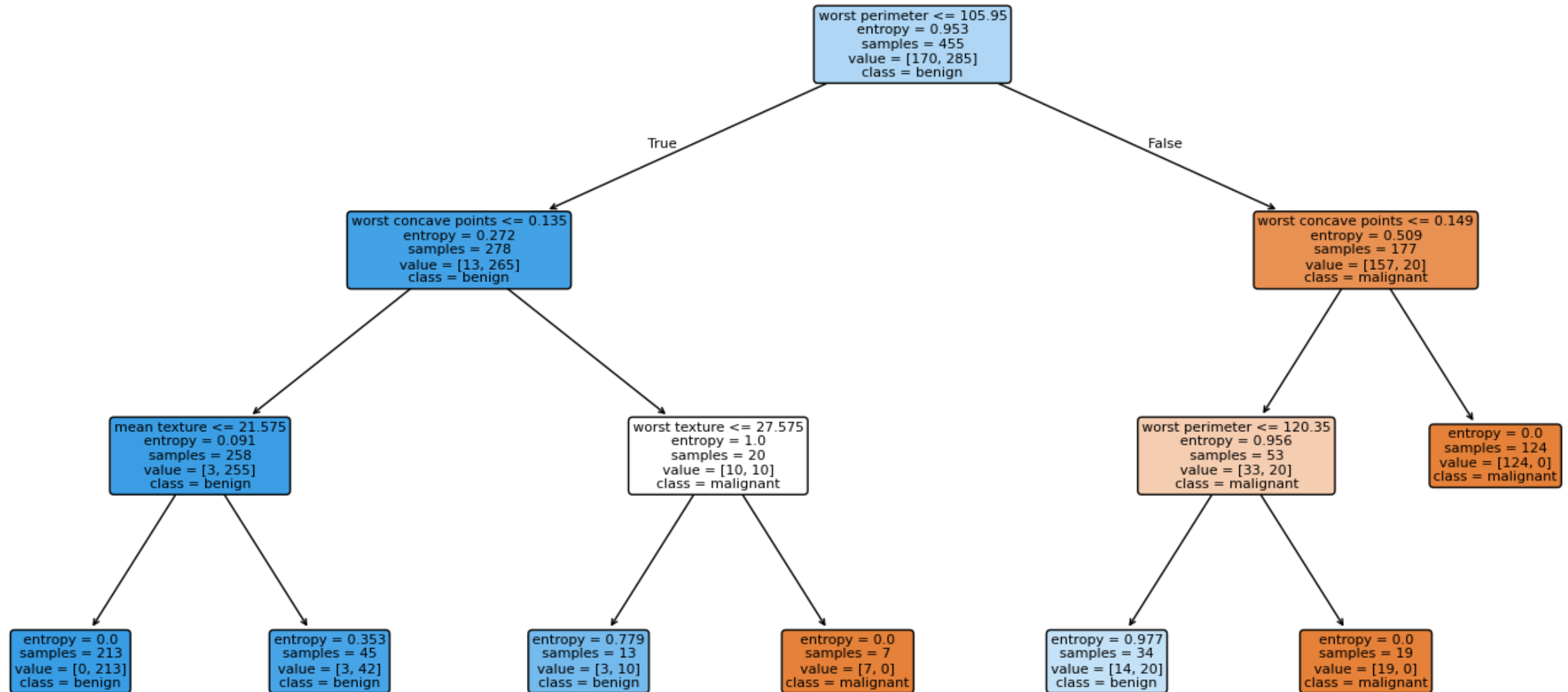
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

fig, ax = plt.subplots(figsize=(14, 7))

plot_tree(
    search.best_estimator_,
    feature_names=X.columns,
    class_names=data.target_names,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8,
    ax=ax
)

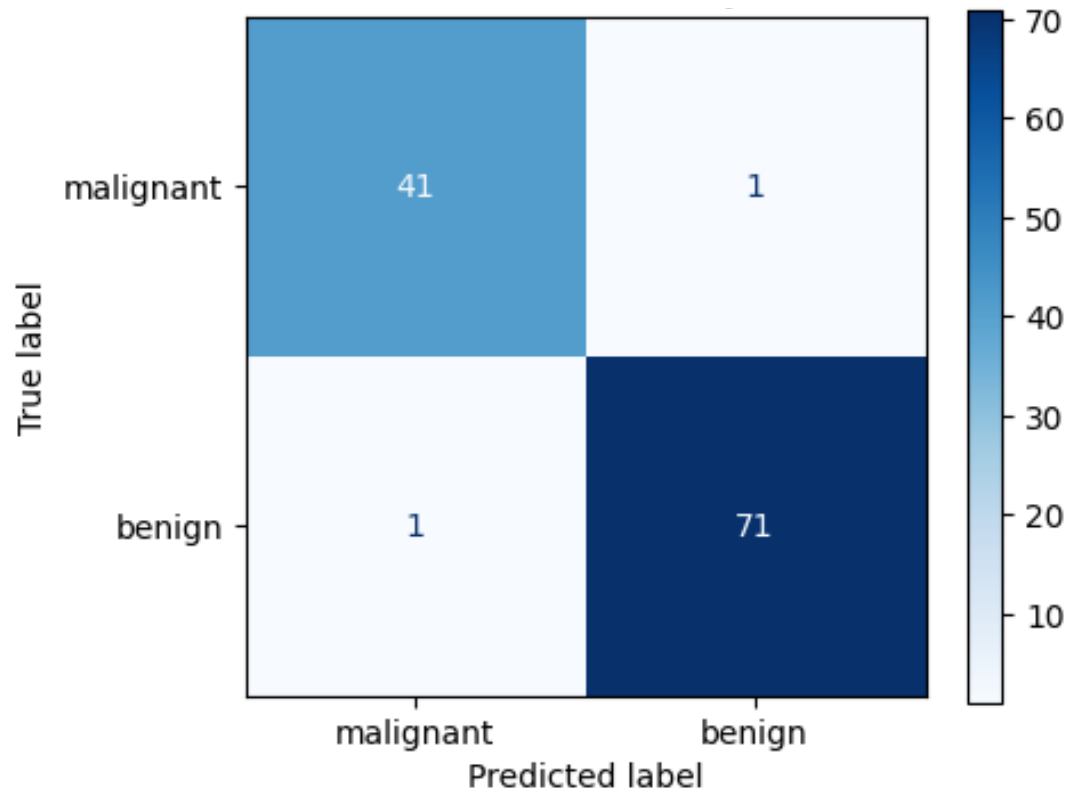
plt.tight_layout()
plt.show()
```

# Selected Decision Tree

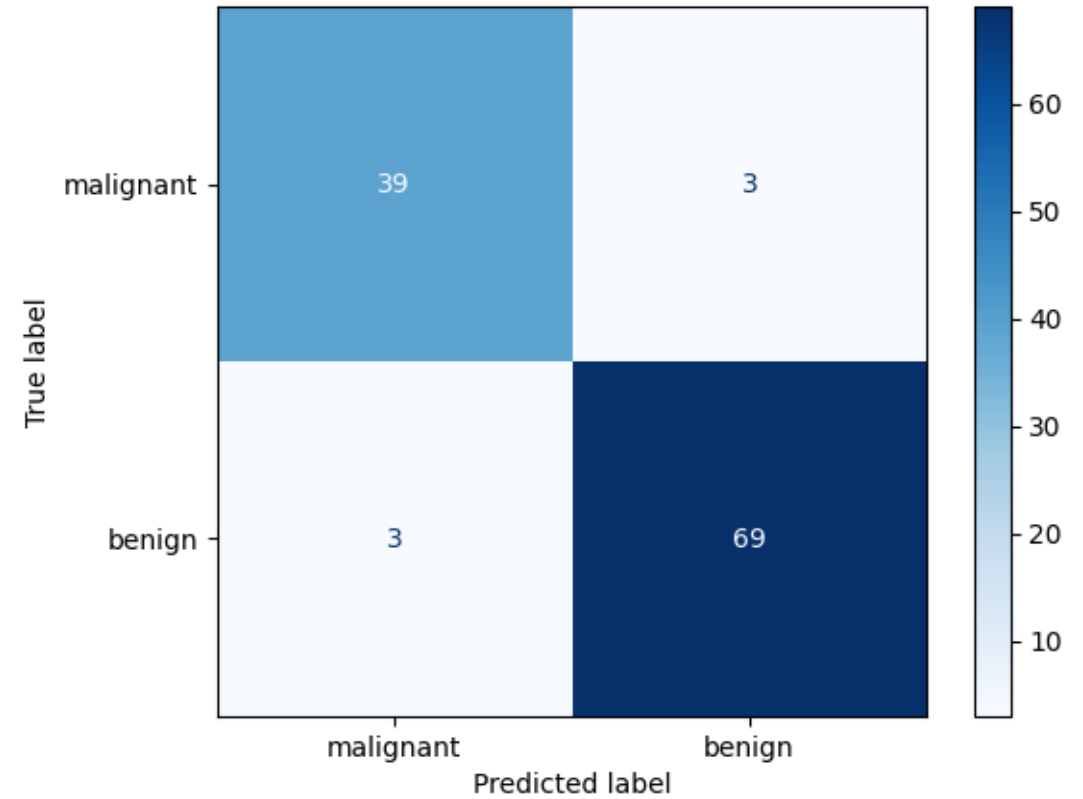


# SVM vs Decision Tree

## SVM



## Decision Tree



**When building a classification decision tree, how is the split at each node selected?**

- A. By gradient descent
- B. By choosing the feature and threshold that give the largest impurity reduction
- C. By randomly selecting a feature and threshold
- D. By minimizing training time

*Answer: B.*

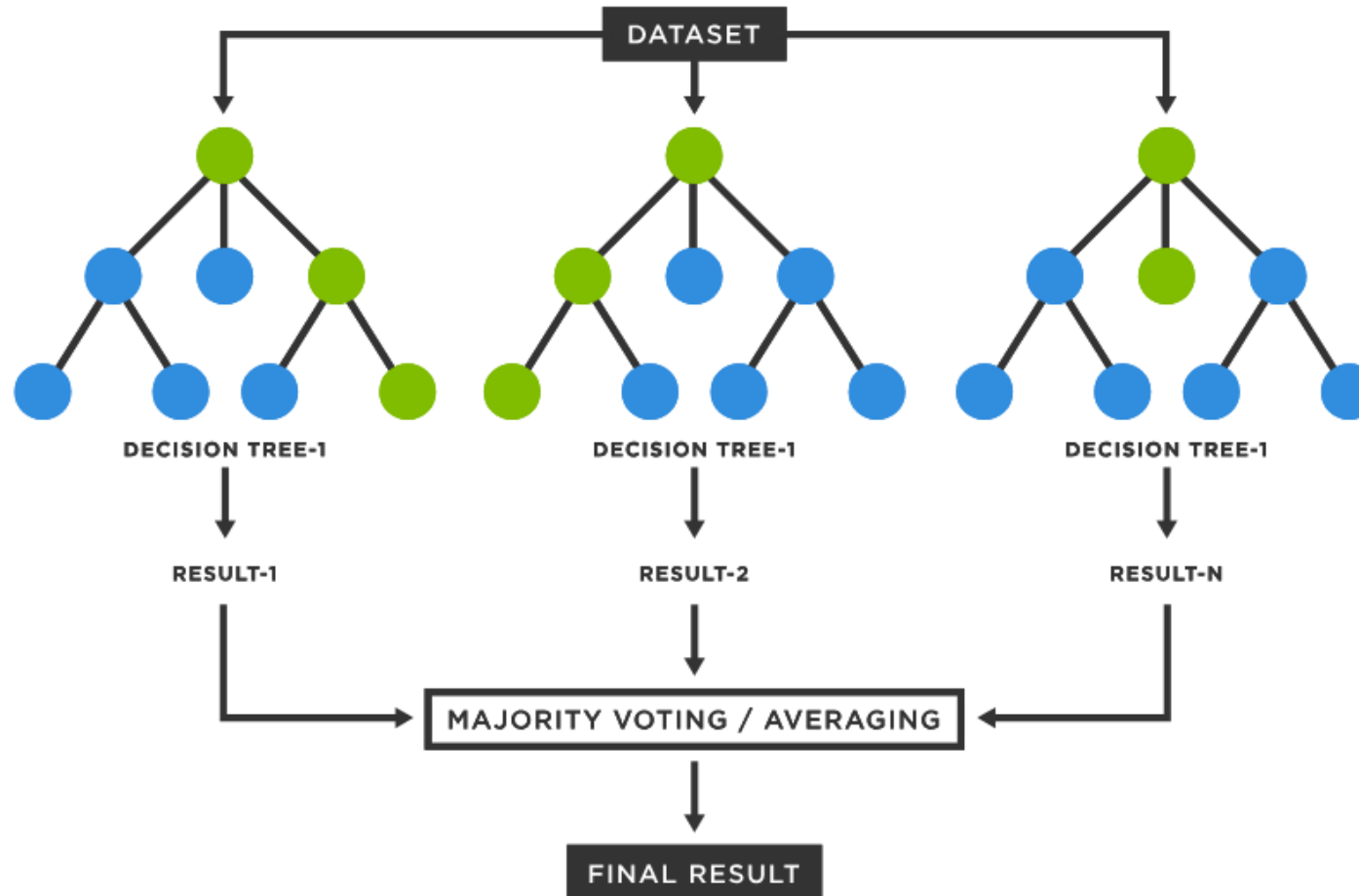
**Which change is most likely to make a decision tree more complex and increase the risk of overfitting?**

- A. Decreasing max\_depth
- B. Increasing min\_samples\_leaf
- C. Increasing max\_depth
- D. Increasing min\_samples\_split

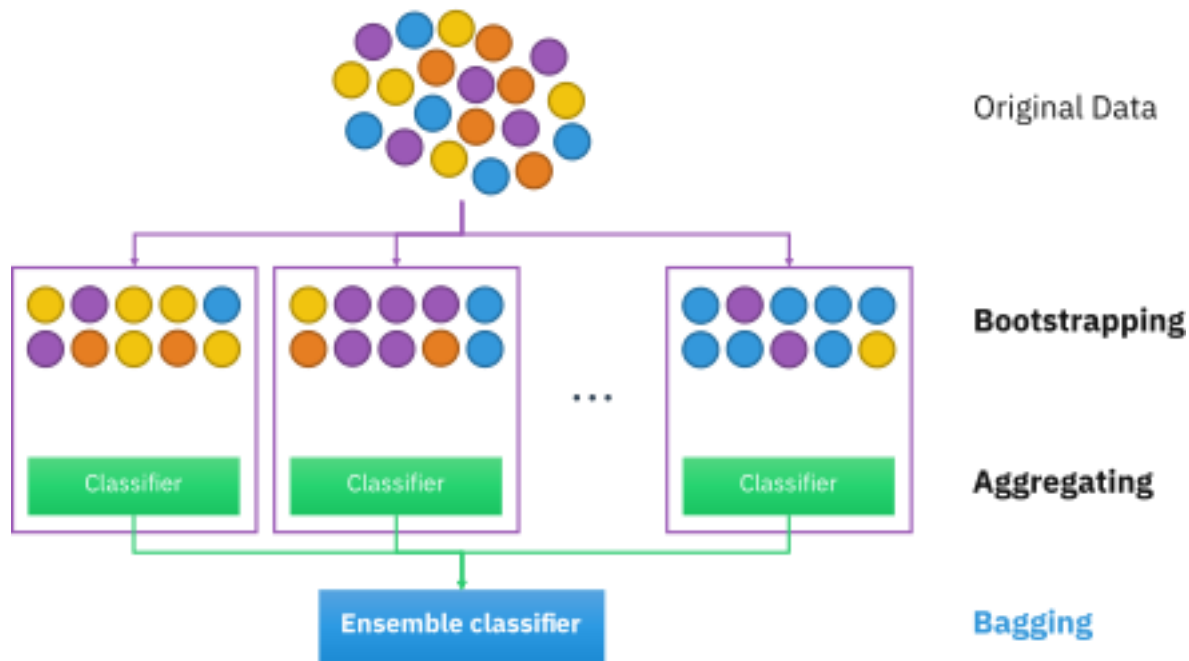
*Answer: C.*

# Random Forest: Many Different Trees, One Prediction

A single decision tree often has high variance, meaning that even a small change in the training data can lead to a very different tree.



# Randomness in a Forest



## 1. Bootstrap the training data

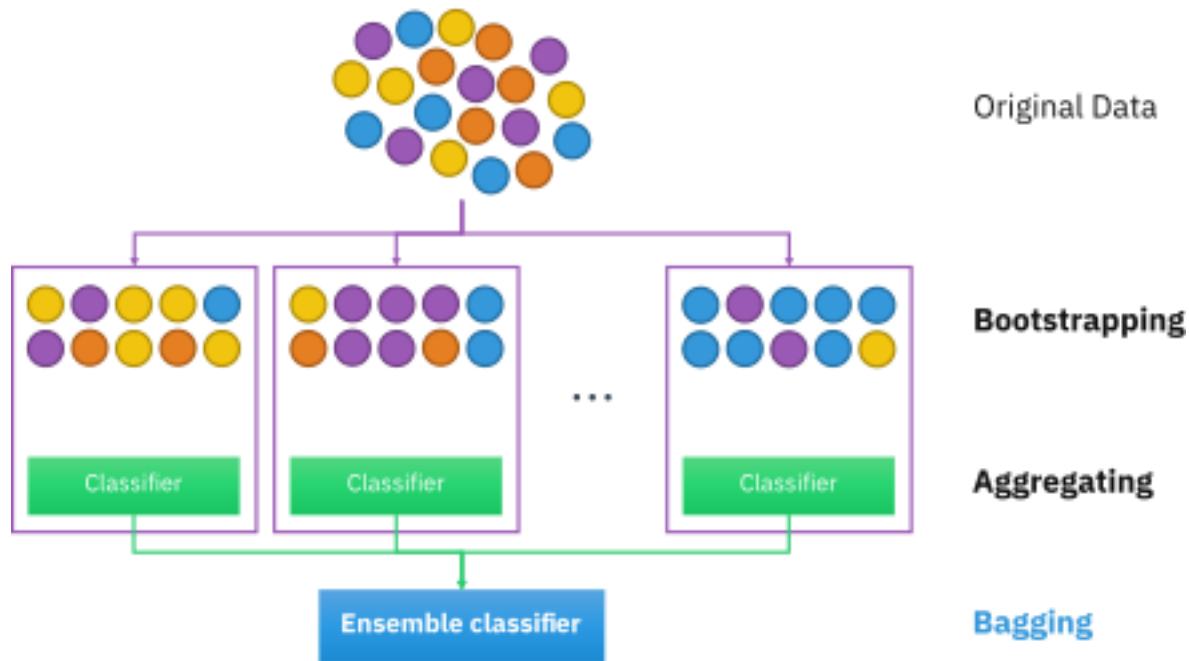
Each tree is trained on a different random sample **with replacement**.

$$D \rightarrow D_1, D_2, D_3, \dots$$

Different trees see slightly different training examples.

The sample size is the same as the original dataset. The same data point can appear multiple times.

# Randomness in a Forest



## 2. Randomly select candidate features at each split

At each node, the tree considers only a **random subset** of features.

This is the key reason “max\_features” matters in Random Forest.

## 3. Combine predictions

Majority voting / average class probability for classification.

**Combining less-correlated trees reduces variance and usually improves generalization compared with a single deep tree.**

# Randomness in a Forest

**Combining less-correlated trees reduces variance and usually improves generalization compared with a single deep tree.**

- When trees are highly correlated, they tend to make the same errors.
- When they are less correlated, some errors are high and others are low, so averaging cancels part of the error.

For example, suppose the correct value is 10:

- Tree 1 predicts 14 and Tree 2 predicts 6. Their errors cancel, and the average is 10.
- Tree 1 predicts 14 and Tree 2 also predicts 14. Averaging still gives 14.

# Random Forest Hyperparameters

## `sklearn.ensemble.RandomForestClassifier`

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,
criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None,
min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None,
random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0,
max_samples=None)
```

[\[source\]](#)

### Parameters:

**`n_estimators` : *int*, *default=100***

The number of trees in the forest.

*Changed in version 0.22:* The default value of `n_estimators` changed from 10 to 100 in 0.22.

**`criterion` : {"gini", "entropy", "log\_loss"}, *default="gini"***

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log\_loss" and "entropy" both for the Shannon information gain, see [Mathematical formulation](#). Note: This parameter is tree-specific.

### Higher `n_estimators`

- more trees → more stable ensemble
- variance usually decreases and then plateaus
- higher computational cost

### Lower `n_estimators`

- fewer trees → noisier / less stable prediction
- higher variance of the ensemble
- faster computation

# Aggregating Results

In `sklearn.RandomForestClassifier`, prediction is actually based on the **highest average class probability across trees**, rather than simply counting hard labels.

|         | P(A) | P(B) |
|---------|------|------|
| Tree 1  | 0.9  | 0.1  |
| Tree 2  | 0.4  | 0.6  |
| Tree 3  | 0.8  | 0.2  |
| Tree 4  | 0.3  | 0.7  |
| Average | 0.6  | 0.4  |

# Hyperparameter Tuning

## 1. Load data and split

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer(as_frame=True)

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=26
)

print(X.shape)
print(len(X_train), len(X_test))

(569, 30)
455 114
```

## 2. 5-fold CV

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(
    random_state=26
)

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, None],
    "max_features": ["sqrt", 0.5, None],
    "min_samples_leaf": [1, 2, 5]
}

search = GridSearchCV(
    rf,
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

search.fit(X_train, y_train)
```

# Hyperparameter Tuning

## 3. Best model and final test

```
from sklearn.metrics import accuracy_score

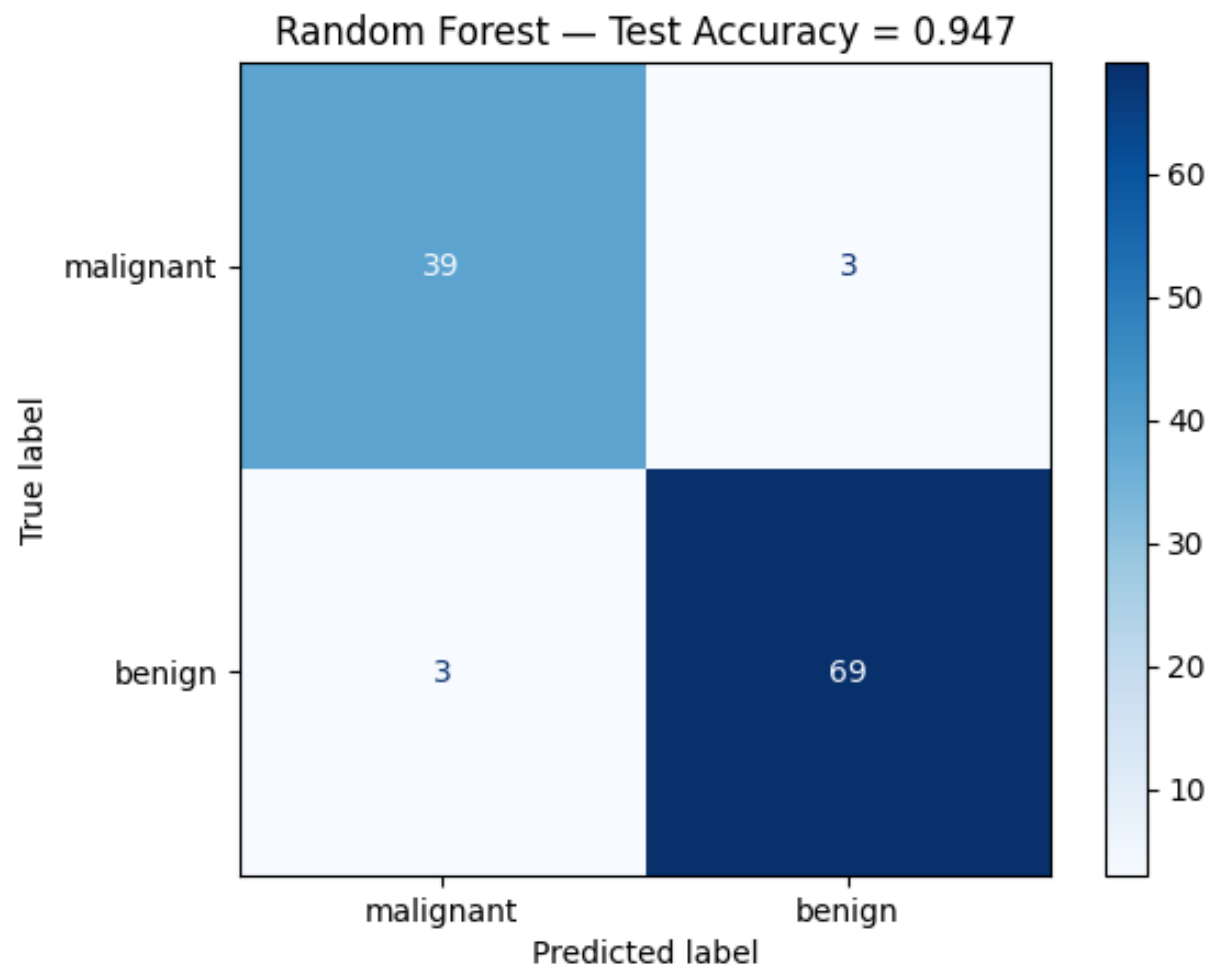
print("Best CV accuracy:", search.best_score_)
print("Best parameters:", search.best_params_)

y_pred = search.predict(X_test)

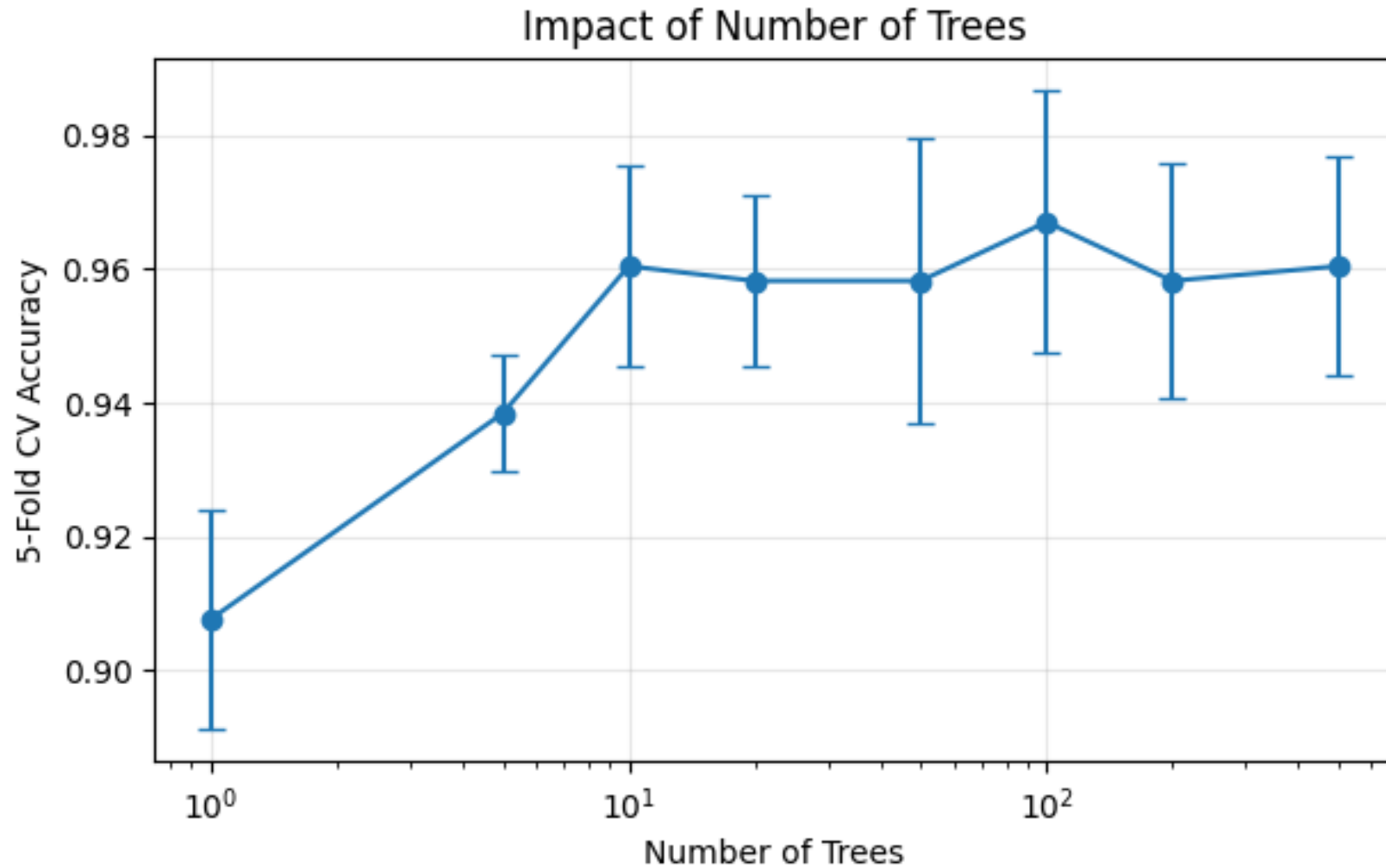
test_acc = accuracy_score(y_test, y_pred)

print("Test accuracy:", test_acc)
```

Best CV accuracy: 0.9670329670329669  
Best parameters: {'max\_depth': None, 'max\_features': 0.5, 'min\_samples\_leaf': 1, 'n\_estimators': 100}  
Test accuracy: 0.9473684210526315



# Number of Trees



**Add enough trees for the prediction to stabilize; beyond that, more trees usually give little additional benefit.**

# Feature Importance

Suppose a node uses feature  $x_j$  to split the data.

Before the split, impurity is  $I_P$ . Afterward:

$$I_{\text{after}} = \frac{|D_L|}{|D_P|} I_L + \frac{|D_R|}{|D_P|} I_R$$

So the impurity reduction at that node is

$$\Delta I = I_P - \frac{|D_L|}{|D_P|} I_L - \frac{|D_R|}{|D_P|} I_R$$

**Feature importance measures how much a feature reduces impurity across the forest.**

For feature  $j$ , one tree sums all reductions from nodes that use that feature:

$$FI_j^{(t)} = \sum_{\text{nodes using } j} \frac{N_P}{N} \Delta I$$

Then Random Forest averages across all  $T$  trees:

$$FI_j = \frac{1}{T} \sum_{t=1}^T FI_j^{(t)}$$

Finally the importances are normalized:

$$\sum_j FI_j = 1$$

# Feature Importance

```
import pandas as pd

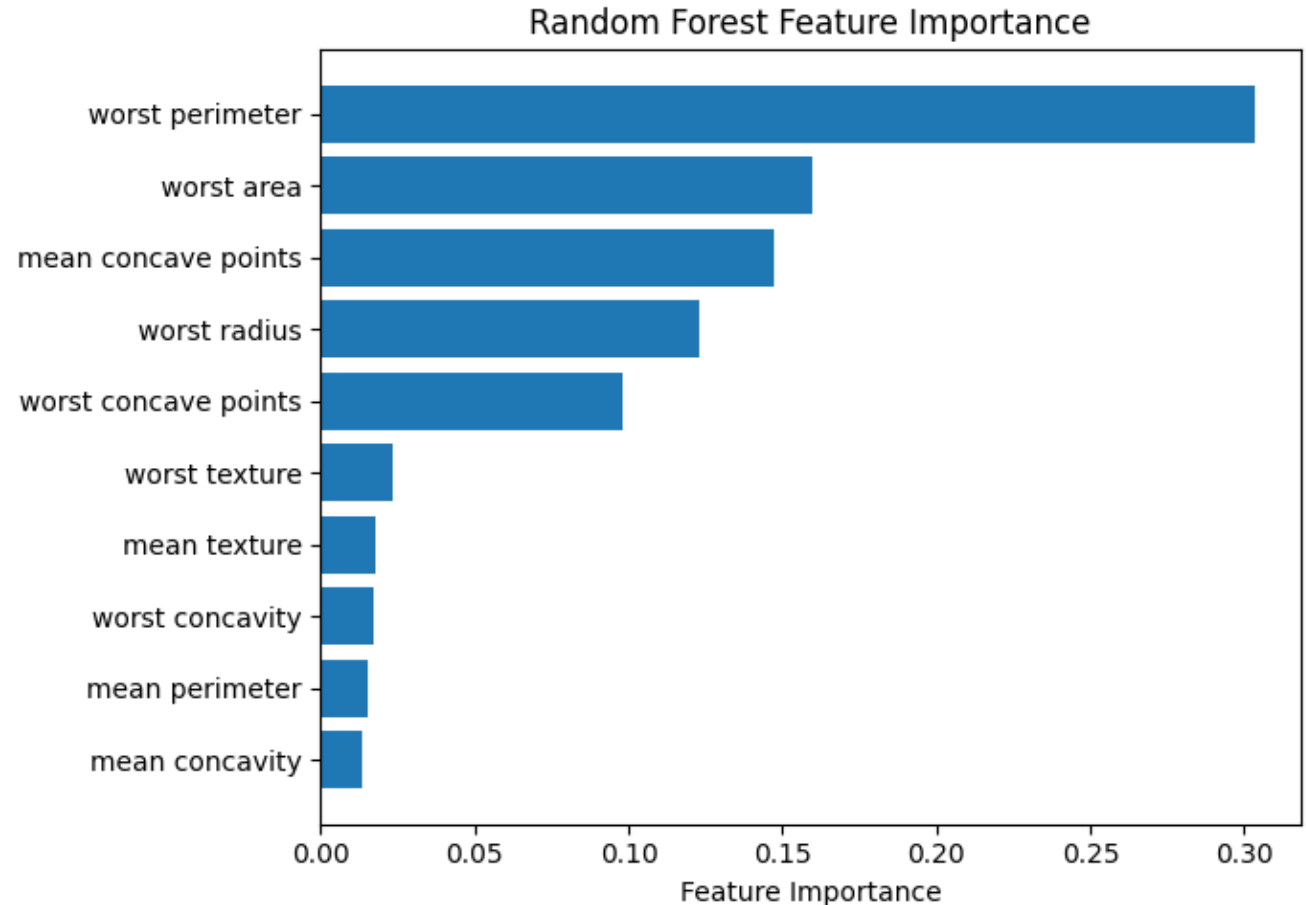
best_rf = search.best_estimator_

importance = pd.Series(
    best_rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print(importance.head(10))
```

|                      |          |
|----------------------|----------|
| worst perimeter      | 0.303219 |
| worst area           | 0.159641 |
| mean concave points  | 0.146983 |
| worst radius         | 0.122940 |
| worst concave points | 0.098348 |
| worst texture        | 0.023537 |
| mean texture         | 0.017626 |
| worst concavity      | 0.016898 |
| mean perimeter       | 0.015369 |
| mean concavity       | 0.013703 |

dtype: float64



**Feature importance describes how the fitted forest used a feature.  
It does not establish causality.**

**Why does a random forest consider only a random subset of features at each split?**

- A. To make every tree identical
- B. To reduce correlation among the trees
- C. To remove the need for bootstrap sampling
- D. To guarantee that each tree is shallow

*Answer: B. Less-correlated trees provide a larger variance reduction when their predictions are averaged.*

**Four trees predict  $P(A)=0.9$ , 0.4, 0.8, and 0.3. What does the random forest predict?**

- A. Class A because the average probability is 0.60
- B. Class B because two trees favor class B
- C. Class A because the largest single probability is 0.90
- D. The forest cannot make a prediction

*Answer: A. The average probabilities are  $P(A)=0.60$  and  $P(B)=0.40$ .*

# Interactive Python Time

`15_random_forest.ipynb`